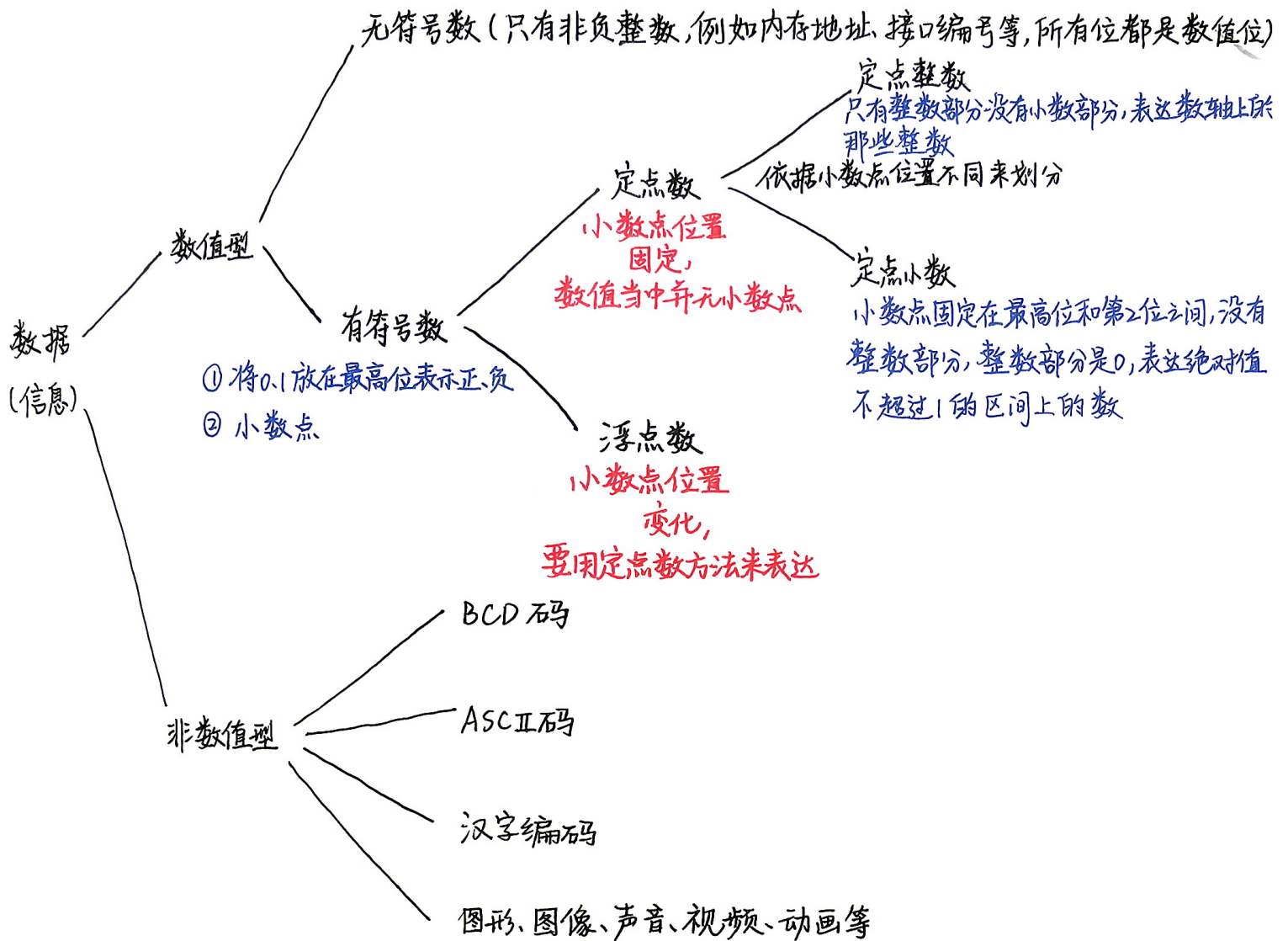


计算机中的信息表示



无符号数

寄存器的位数反映了无符号数的表示范围

例如, 8位可表示 $0 \sim 2^8 - 1$, 共 2^8 个数; 16位可表示 $0 \sim 2^{16} - 1$, 共 2^{16} 个数

$$n = 2^n - 1$$

255

65535

有符号数

注意! 将真值表达成机器数时, 长度不可随意, 与机器的机器字长有关

因此, 就要采用编码了: 原/补/反/移码机器数

1. 机器数与真值

计算机能表达的
二进制的长度

以8位为例

(与寄存器位数有关)

通常与机器数长度一致

真值 (带符号的数)

+0.1011

-0.1011

+1100

-1100

机器数 (符号数字化的数)

(符号位) 最高位

0 1011

1 1011

0 1100

1 1100

• 小数点位置

没有小数部分

2. 原码表示法

整数

数位是n位的机器数表示范围: $-2^n \sim 2^n$

小数

$$[x]_{\text{原}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^n - x & 0 \geq x > -2^n \end{cases}$$

不包括符号位

其中, x 为真值, n 为整数的位数

例如:

$$x = +1100$$

4位

$$[x]_{\text{原}} = 0, 1100$$

4位

$$x = -1100$$

4位

$$[x]_{\text{原}} = 2^4 + 1100 = 1, 1100$$

仅差一个符号位

正数加“0,”
负数加“1,”

用逗号将符号位和数值部分隔开

只是一种书写形式, 在计算机中实际不存在

例1. 已知 $[x]_{\text{原}} = 1.0011$, 求 x

$$\because [x]_{\text{原}} = 1-x$$

$$\therefore x = 1 - [x]_{\text{原}} = 1 - 1.0011 = -0.0011$$

例2. 已知 $[x]_{\text{原}} = 1, 1100$, 求 x

$$\because [x]_{\text{原}} = 2^4 - x$$

$$\therefore x = 2^4 - [x]_{\text{原}} = 10000 - 1, 1100 = -1100$$

例3. 已知 $[x]_{\text{原}} = 0.1101$, 求 x

$$\because [x]_{\text{原}} = x$$

$$\therefore x = +0.1101$$

例如:

$$x = +0.1101$$

$$[x]_{\text{原}} = 0.1101$$

$$x = -0.1101$$

$$[x]_{\text{原}} = 1 - (-0.1101) = 1.1101$$

$$x = +0.1000000$$

$$[x]_{\text{原}} = 0.1000000$$

$$x = -0.1000000$$

$$[x]_{\text{原}} = 1 - (-0.1000000) = 1.1000000$$

用小数点将符号位和数值部分隔开

例4. 求 $x=0$ 的原码

$$\left\{ \begin{array}{l} \text{设 } x = +0000, \text{ 则 } [x]_{\text{原}} = 0.0000 \\ \text{设 } x = -0000, \text{ 则 } [x]_{\text{原}} = 2^4 + 0000 = 1.0000 \end{array} \right.$$

$$\text{设 } x = +0.0000, \text{ 则 } [x]_{\text{原}} = 0.0000$$

$$\text{设 } x = -0.0000, \text{ 则 } [x]_{\text{原}} = 1 - (-0.0000) = 1.0000$$

$$[+0]_{\text{原}} \neq [-0]_{\text{原}}$$

$$[+0.0000]_{\text{原}} \neq [-0.0000]_{\text{原}}$$

原码的特点：简单、直观

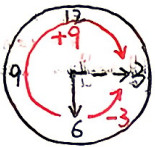
但用原码作加法时，会出现以下问题：

要求	数1	数2	实际操作	结果符号
加	正	正	加	正
加	正	负	减	正/负
加	负	正	减	正/负
加	负	负	加	负

要想只做法，就要找到一个与负数等价的正数来代替该负数，使减法转化为加法。
是该负数的补数

3. 补码表示法

概念引入



逆时针 $6-3=3$

顺时针 $6+9=15$ $15-12=3$
模

例如， $-3 \equiv +9 \pmod{12}$

$-4 \equiv +8 \pmod{12}$

$-5 \equiv +7 \pmod{12}$

-3 可以用 +9 代替，使减法 → 加法
+9 是 -3 以 12 为模的补数

负数 + 模 = 该负数的补数

当一正数与一负数互为补数时，其绝对值之和为模

例如。

计数器 (模 16)

如何归零

$1011 \xrightarrow{?} 0000$

$$\begin{array}{r} 1011 \\ -1011 \\ \hline 0000 \end{array}$$

$$\begin{array}{r} 1011 \\ +0101 \\ \hline 10000 \end{array}$$

在计算机当中，由于机器数长度的限制，是有表示范围的，
模是机器数表示的状态数，
这个范围与二进制位数有关。
机器字长越大，模越大

可见 -1011 能用 $+0101$ 代替，记作 $-1011 \equiv +0101 \pmod{2^4}$

同理： $-011 \equiv +101 \pmod{2^3}$

$-0.1001 \equiv +1.0111 \pmod{2}$

$$\begin{array}{r} 0.1001 \\ -0.1001 \\ \hline 0.0000 \end{array} \quad \begin{array}{r} 0.1001 \\ +1.0111 \\ \hline 10.0000 \end{array}$$

定点小数的模始终是 2，与长度无关



该位的位权 2^1 就是模

$x < 0$ 时：

$x_{补} = 模 + x$

$|x| + x_{补} = 模$

$x_{补} = 模 - |x|$

$x_{补} = 模 + x$

正数的补数即为其本身

$\therefore -1011 \equiv +0101 \pmod{2^4}$

两个互为补数的数分别加上模，结果仍互为补数

$$\begin{array}{r} -1011 \\ +10000 \\ \hline +0101 \end{array} \quad \begin{array}{r} +0101 \\ +10000 \\ \hline +10101 \end{array}$$

寄存器位数是 4 位
由此可见，模 16 运算下，

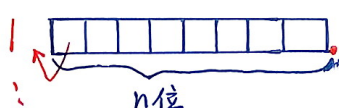
-11 的补数是 5，5 的补数也是 5

那么，如果想区分这个补码是正数的补码还是负数的补码，
就要加上 2^{n+1} ，其中 n 表示数值的位数，现用寄存器位数是 5 位

$$\begin{array}{r} -1011 \\ +100000 \\ \hline +10101 \end{array} \quad \begin{array}{r} +0101 \\ +100000 \\ \hline +10101 \end{array}$$

可以使用这样的一位来表示补码是正数还是负数

对于模数计算器，机器数能表示的状态数
就是它的模 (2^n)



该位的位权 2^n 就是模

整数

对正数来说,原码=补码,不需要变

$$[x]_{\text{补}} = \begin{cases} 0, x & \text{范围 } 2^n \rightarrow x \geq 0 \\ 2^{n+1} + x & \text{不对称 } 0 \rightarrow x \geq -2^n \pmod{2^{n+1}} \end{cases}$$

(n+1)是整个机器数的长度,8位长的话n就是7

其中, x 为真值, n 为整数的位数

小数

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 \rightarrow x \geq -1 \pmod{2} \end{cases}$$

其中, x 为真值

例如:

$$x = +1010 \quad [x]_{\text{补}} = 0, 1010$$

$$x = -1011000 \quad [x]_{\text{补}} = 2^{7+1} + (-1011000) \\ = 100000000 \\ - 1011000 \\ \hline 0101000$$

可代表真值符号,但不可用其

比较真值大小

用逗号将符号位和数值部分隔开

例如:

$$x = +0.1110 \quad [x]_{\text{补}} = 0.1110$$

$$x = -0.1100000 \quad [x]_{\text{补}} = 2 + (-0.1100000) \\ = 10.0000000 \\ - 0.1100000 \\ \hline 1.0100000$$

只是找到一个等效的正数,

不能看出真值大小

用小数点将符号位和数值部分隔开

补码的快捷求法

整数:

$$[x]_{\text{原}} = 1, x_1 x_2 \dots x_n$$

$$[x]_{\text{补}} = 1, \bar{x}_1 \bar{x}_2 \dots \bar{x}_n + 1$$

小数:

$$[x]_{\text{原}} = 1, x_1 x_2 \dots x_n$$

$$[x]_{\text{补}} = 1, \bar{x}_1 \bar{x}_2 \dots \bar{x}_n + 2^{-n}$$

$$\text{若 } x = -1010, \text{ 则 } [x]_{\text{补}} = 2^{4+1} - 1010 = 11111 + 1 - 1010$$

数值位4位,总长5位

$$\begin{array}{r} 10000 \\ - 1010 \\ \hline 10110 \end{array} \quad \begin{array}{r} 11111 \\ - 1010 \\ \hline 10101 + 1 \\ \hline 10110 \end{array}$$

$$\text{又 } [x]_{\text{原}} = 1, 1010$$

对于整数而言,当真值为负时,补码

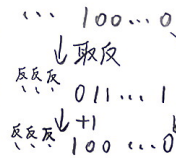
可用原码除符号位外每位取反,末位加1求得

小妙招

已知 x 原码, 符号位不变, 数值位从右往左, 第一个“1”以后取反, 得 [x]补 (仅针对负数)

$$\text{例如, } [x]_{\text{原}} = 1, 1100000 \rightarrow [x]_{\text{补}} = 1, 0100000$$

$$[x]_{\text{原}} = 1, 1011000 \rightarrow [x]_{\text{补}} = 1, 0101000$$



其实相当于上面方法

练习:

① 已知 $[x]_{\text{补}} = 0.0001$, 求 x

解: 由定义得 $x = +0.0001$

若机器数长度更长, 就要在前面添0

② 已知 $[x]_{\text{补}} = 1.0001$, 求 x

解: 由定义得 $x = [x]_{\text{补}} - 2$
 $= 1.0001 - 10.0000$
 $= -0.1111$

③ 已知 $[x]_{\text{补}} = 1.1110$, 求 x

解: 由定义得 $x = [x]_{\text{补}} - 2^{4+1}$
 $= 1.1110 - 100000$
 $= -0010$

或: 由 $[x]_{\text{补}} = 1.1110$ 知 $[x]_{\text{原}} = 1, 0010$
 故 $x = -0010$

真值: 既不能改变其大小, 又要让每一位都有确定的值

x	$[x]_{\text{补}}$	$[x]_{\text{原}}$
$x = +70 = 1000110$	$0, 1000110$	$0, 1000110$
$x = -70 = -1000110$	$1, 0111010$	$1, 1000110$
$x = 0.1110$	0.1110	0.1110
$x = -0.1110$	1.0010	1.1110
$x = 0.0000$	0.0000	0.0000
$x = -0.0000$	0.0000	0.0000
$x = -1.0000$	1.0000	1.0000



补码比原码多表示一个数, 因为0的补码只有一种形式, 是0.0000, 唯一的, 在原码中的另一种形式就可以拿不出来表示一个数

$$[-1]_{\text{补}} = 2 + x \\ = 10.0000 - 1.0000 \\ = 1.0000$$

定点小数的原码不能表示-1而补码可以

用小数的形式来表达是表达不了的, 而用整数的形式来表达是可以得出原码的

$$[-1]_{\text{原}} = 1, 0001$$

假设数值位是4位

反码

整数

$$[x]_{\text{反}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ (2^{n+1}-1) + x & 0 \geq x > -2^n \pmod{2^{n+1}-1} \end{cases}$$

x 为真值, n 为整数的位数

例如

$$x = +1101 \quad [x]_{\text{反}} = 0, 1101$$

$$x = -1101 \quad [x]_{\text{反}} = (2^{4+1}-1) - 1101$$

对于负数, 符号位不变, 数值位全部反过来

$$= 11111 - 1101$$
$$= 1, 0010$$

用逗号将符号位与数值部分隔开

例如:

已知 $[x]_{\text{反}} = 0, 1110$, 求 x .

解: 由定义得, $x = +1110$

已知 $[x]_{\text{反}} = 1, 1110$, 求 x .

解: 由定义得, $x = [x]_{\text{反}} - (2^{4+1}-1)$

$$= 1, 1110 - 11111$$
$$= -0001$$

小数

$$[x]_{\text{反}} = \begin{cases} x & 1 > x \geq 0 \\ (2-2^n) + x & 0 \geq x > -1 \pmod{2-2^{-n}} \end{cases}$$

x 为真值, n 为小数的位数

例如,

$$x = +0.1101 \quad [x]_{\text{反}} = 0.1101$$

$$x = -0.1010 \quad [x]_{\text{反}} = (2-2^{-4}) + (-0.1010)$$
$$= (10.0000 - 0.0001) - 0.1010$$
$$= 1.1111 - 0.1010$$
$$= 1.0101$$

用小数点将符号位与数值部分隔开

求 0 的反码

解: 设 $x = +0.0000$, $[+0.0000]_{\text{反}} = 0.0000$

$$\text{设 } x = -0.0000, [-0.0000]_{\text{反}} = 2 - 2^{-4} - 0.0000$$
$$= (10.0000 - 0.0001) - 0.0000$$
$$= 1.1111 - 0.0000 = 1.1111$$

设 $x = +0$, $[+0]_{\text{反}} = 0.0000$

设 $x = -0$, $[-0]_{\text{反}} = (2^{4+1}-1) - 0000 = 11111 - 0000 = 1, 1111$

因此 $[+0]_{\text{反}} \neq [-0]_{\text{反}}$

三种机器数小结

- 最高位是符号位, 书写上用“,” (整数) 或 “.” (小数) 将数值部分和符号位隔开
- 对于正数, 原码 = 补码 = 反码
- 对于负数, 符号位是 1, 其数值部分:

原码 $\rightarrow$ 补码: 除符号位每位取反末位加 1

原码 $\rightarrow$ 反码: 除符号位外每位取反

设机器数字长是8位,其中有1位是符号位,对于整数,当其分别表示无符号数、原码、补码和反码时,对应的真值范围各是多少?

二进制代码	无符号数 对应的真值	原码 对应的真值	补码 对应的真值	反码 对应的真值
00000000	0	+0	± 0	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0

不论定点整数或定点小数

其能表示的绝对值最大

的负数都是这种形式

对补码:

既可表示真值的符号

又可表示真值的值

$-2^7 = -128$ 负号 位权: 2^7

已知 $[y]_{\text{补}}$, 求 $[-y]_{\text{补}}$.

变补运算

正负都是这个规律

对补码!

设 $[y]_{\text{补}} = y_0 y_1 y_2 \dots y_n$

当 y_0 为 0 时: $[y]_{\text{补}} = 0 y_1 y_2 \dots y_n$

$y = 0 y_1 y_2 \dots y_n + 1$

$-y = -0 y_1 y_2 \dots y_n$

$[-y]_{\text{补}} = 1 \bar{y}_1 \bar{y}_2 \dots \bar{y}_n + 2^{-n}$

这里符号位不变

当 y_0 为 1 时: $[y]_{\text{补}} = 1 y_1 y_2 \dots y_n$

$[y]_{\text{原}} = [y]_{\text{补}} + 1 = 1 \bar{y}_1 \bar{y}_2 \dots \bar{y}_n + 1$

$y = -(0 \bar{y}_1 \bar{y}_2 \dots \bar{y}_n + 2^{-n})$

$-y = 0 \bar{y}_1 \bar{y}_2 \dots \bar{y}_n + 2^{-n}$ 正的

$[-y]_{\text{补}} = 0 \bar{y}_1 \bar{y}_2 \dots \bar{y}_n + 2^{-n}$

连同符号位在内,每位取反,末位加1

移码表示法

补码表示很难直接判断其真值大小

比如:

十进制	二进制	补码
$x=+2$	$+1010$	$0,1010$
$x=-2$	-1010	$1,0101$
$x=+3$	$+1111$	$0,1111$
$x=-3$	-1111	$1,0000$

不可比较

不可比较

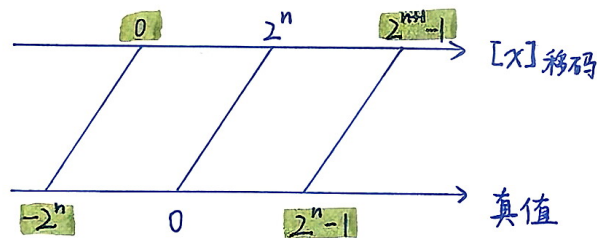
$x+2^5$: 加的一定不能少于 2^n
 要把所有数据都平移到 x 的正半轴, 所以加 2^5 而非 2^4
 ≥ 0
 偏移量不够
 都移到非负区域, 方便比较大小

$+1010 + 100000 = 110101$	可比较
$-1010 + 100000 = 001011$	可比较
$+1111 + 100000 = 111111$	可比较
$-1111 + 100000 = 000001$	可比较

移码定义

$$[x]_{\text{移}} = 2^n + x \quad (2^n > x \geq -2^n)$$

x 为真值, n 为整数的位数



例如,

$$x = 10100, [x]_{\text{移}} = 2^5 + 10100 = 1,10100$$

$$x = -10100, [x]_{\text{移}} = 2^5 - 10100 = 0,01100$$

用逗号将符号位和数值部分隔开
 不要把移码当作带符号的数!

移码与补码的比较

设 $x = +1100100$

$$[x]_{\text{移}} = 2^7 + 1100100 = 1,1100100$$

$$[x]_{\text{补}} = 0,1100100$$

绝对值最大的负数
 -2^n

设 $x = -1100100$

$$[x]_{\text{移}} = 2^7 - 1100100 = 0,0011100$$

$$[x]_{\text{补}} = 1,0011100$$

通常用来表示浮点数的阶
 尤指整数的

补码与移码只差一个符号位!

真值 $x(n=5)$	$[x]_{\text{补}}$	$[x]_{\text{移}}$	$[x]_{\text{移}}$ 对应的十进制整数
-10000	100000	000000	0
-11111	100001	000001	1
-11110	100010	000010	2
$\vdots$	$\vdots$	$\vdots$	$\vdots$
-00001	111111	011111	31
± 00000	000000	100000	32
$+00001$	000001	100001	33
$+00010$	000010	100010	34
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$+11110$	011110	111110	62
$+11111$	011111	111111	63

$$\text{当 } x=0 \text{ 时: } [+0]_{\text{移}} = 2^5 + 0 = 1,00000$$

$$[-0]_{\text{移}} = 2^5 - 0 = 1,00000$$

$$[+0]_{\text{移}} = [-0]_{\text{移}}$$

以 $n=5$ 时为例, 最小真值为 $-2^5 = -100000$

$$[-100000]_{\text{移}} = 2^5 - 100000 = 000000$$

可见, 最小真值的移码是 (全 0)

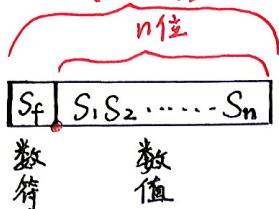
用移码表示浮点数的阶码

能方便地判断浮点数的阶码大小

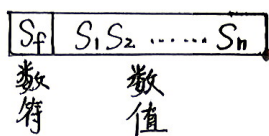
数的定点表示和浮点表示

小数点按约定方式标出

定点表示



或



定点机

小数定点机

整数定点机

原码

$$-(1-2^{-n}) \sim +(1-2^{-n})$$

$$-(2^n-1) \sim +(2^n-1)$$

补码

$$-1 \sim +(1-2^{-n})$$

$$-2^n \sim +(2^n-1)$$

反码

$$-(1-2^{-n}) \sim +(1-2^{-n})$$

$$-(2^n-1) \sim +(2^n-1)$$

(整数)可正可负

浮点表示

浮点数的-般形式

$$N = S \times r^j$$

阶码 "x" "r" 不表达, r-般是2.

可正可负
小数
尾数

计算机中: S 小数, 可正可负; j 整数, 可正可负
(可取2, 4, 8, 16等)

为保证精度最高

当 r=2 时,

$$N = 11.0101$$

$$= 0.110101 \times 2^{10}$$

$$= 1.10101 \times 2^1$$

$$= 1101.01 \times 2^{-10}$$

$$= 0.00110101 \times 2^{100}$$

二进制表示

规格化数

"二者相比, 这个更好, 因为它小数点后第1位就是有效数字, 这样小数点后可有23个有效数字."

是用原码表示的规格化

对正数, 不论原码或补码, 都应是 "0.1..." 整数部分都应是0, 小数点后最高位是1。

要求: ① 绝对值小于1

② 小数点后最高位不能为0

若阶为2, 小数点后最高位必须为1

拓展: 若基数为4, 小数点后2位不全为0

8 3
16 4

j 和 m: 共同表示小数点实际位置

不可兼得

若阶码长, 则该浮点数表示范围大

若尾数长, 则该浮点数的精度高

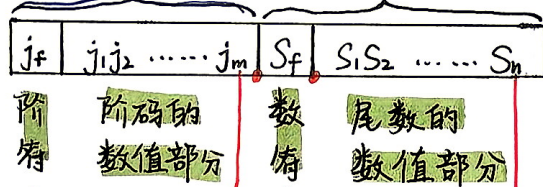
浮点数的表示形式

0.110101 x 2¹⁰ 如何用浮点数表达呢?

(假设阶码长度是8)

00000010 0110101...0

定点整数 j 阶码 S 尾数



其正负表示小数点移动方向

m: 位数反映浮点数表示范围

Sf: 浮点数的符号

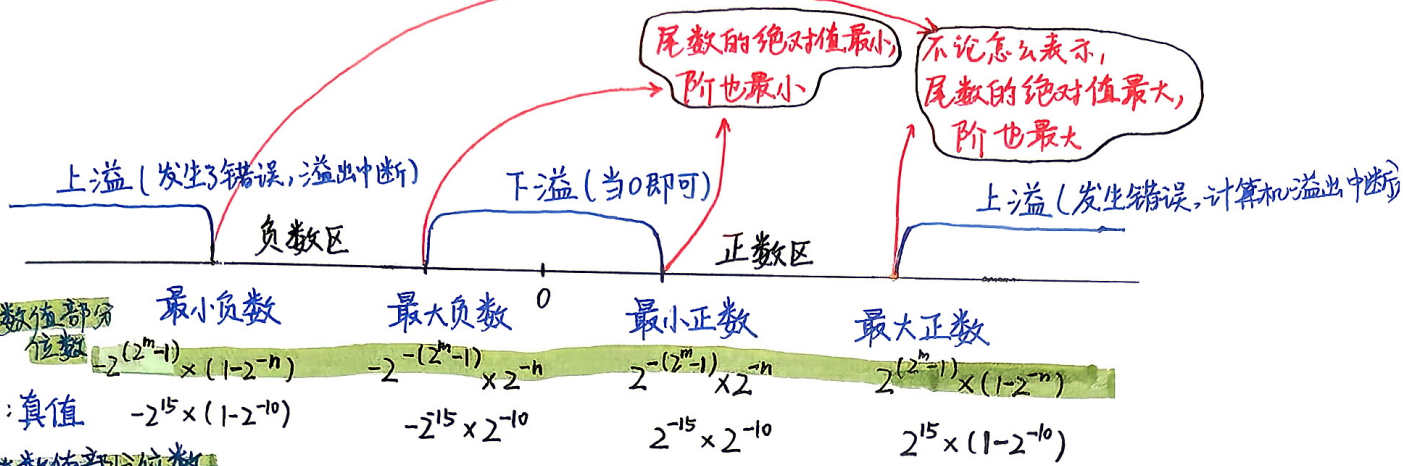
n: 位数反映浮点数精度

浮点数的表示范围 (比定点数的大得多, 但并非无穷大)

上溢 阶码 > 最大阶码

下溢 阶码 < 最小阶码

按机器零处理



设机器数字长是24位, 欲表示±3万的十进制数, 试问在保证数的最大精度的前提下, 除阶符、数符各取1位外, 阶码、尾数各取几位?

解: $\because 2^{14} = 16384$, $2^{15} = 32768$ $\therefore 15$ 位二进制数可以反映±3万之间的十进制数

$2^{15} \times 0. \underbrace{\text{xxx} \dots \text{xxx}}_{15 \text{ 位}}$

又: 题目要求保证数的最大精度 $\therefore$ 可取 $m=4$, $n=18$

浮点数的规格化形式

$r=2$ 尾数最高位是1

基数不同,

$r=4$ 尾数最高2位不全为0

浮点数的规格化形式不同

$r=8$ 尾数最高3位不全为0

浮点数的规格化

基数 r 越大,
可表示的浮点数的范围越大
浮点数的精度越低

$r=2$ $\left\{ \begin{array}{l} \text{左规} \quad \text{尾数左移1位, 阶码减1} \\ \text{右规} \quad \text{尾数右移1位, 阶码加1} \end{array} \right.$

$r=4$ $\left\{ \begin{array}{l} \text{左规} \quad \text{尾数左移2位, 阶码减1} \\ \text{右规} \quad \text{尾数右移2位, 阶码加1} \end{array} \right.$

$r=8$ $\left\{ \begin{array}{l} \text{左规} \quad \text{尾数左移3位, 阶码减1} \\ \text{右规} \quad \text{尾数右移3位, 阶码加1} \end{array} \right.$

例如. 设 $m=4, n=10, r=2$

尾数规格化后的浮点数表示范围

最大正数	$2^{+1111} \times 0.1111111111$	$= 2^{15} \times (1-2^{-10})$	$2^{2^m-1} \times (1-2^{-n})$
最小正数	$2^{-1111} \times 0.1000000000$	$= 2^{-15} \times 2^{-1}$	$2^{-(2^m-1)} \times 2^{-1}$
最大负数	$2^{-1111} \times (-0.1000000000)$	$= -2^{-15} \times 2^{-1}$	$-2^{-(2^m-1)} \times 2^{-1}$
最小负数	$2^{1111} \times (-0.1111111111)$	$= -2^{15} \times (1-2^{-10})$	$-2^{2^m-1} \times (1-2^{-n})$

例. 将 $+\frac{19}{128}$ 写成二进制定点数、浮点数及在定点机和浮点机中的机器数形式, 其中数值部分均取10位, 数符取1位, 浮点数阶码取5位 (含1位阶符).

解: 设 $x = +\frac{19}{128}$

二进制形式 $x = 0.0010011$

定点表示 $x = 0.0010011000$

浮点规格化形式 $x = 0.1001100000 \times 2^{-10}$

定点机中 $[x]_{\text{原}} = [x]_{\text{补}} = [x]_{\text{反}} = 0.0010011000$

浮点机中 $[x]_{\text{原}} = 1, 0010; 0.1001100000$

$[x]_{\text{补}} = 1, 1110; 0.1001100000$ 正的不用变

$[x]_{\text{反}} = 1, 1101; 0.1001100000$

例. 将 -58 表示成二进制定点数和浮点数, 并写出它在定点机和浮点机中的三种机器数及阶码为移码、尾数为补码的形式, 其他要求同上.

解: 设 $x = -58$

二进制形式 $x = -111010$

定点表示 $x = -0000111010$

浮点规格化形式 $x = -(0.1110100000) \times 2^{110}$

定点机中 $[x]_{\text{原}} = 1, 0000111010$

$[x]_{\text{补}} = 1, 1111000110$

$[x]_{\text{反}} = 1, 1111000101$

浮点机中 $[x]_{\text{原}} = 0, 0110; 1, 1110100000$

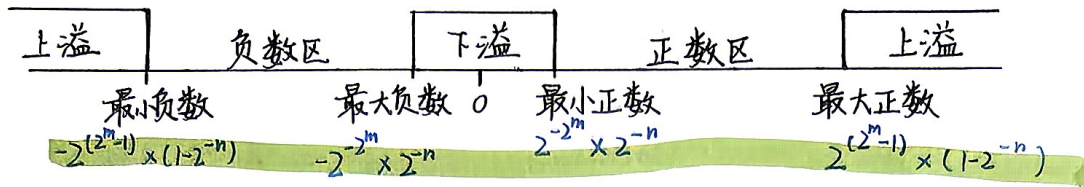
$[x]_{\text{补}} = 0, 0110; 1, 0001100000$

$[x]_{\text{反}} = 0, 0110; 1, 0001100000$ *注意

$[x]_{\text{阶码补}} = 1, 0110; 1, 0001100000$

与补码只差一个符号位!

例. 写出对应下图所示的浮点数的补码形式, 设 $n=10, m=4$, 阶符、数符各取1位



	真值	补码
最大正数	$0,1111 \quad 2^{15} \times (1-2^{-10})$	$0,1111; 0.11111111$
最小正数	$1,0000 \quad 2^{-16} \times 2^{-10}$	$1,0000; 0.0000000001$
最大负数	$-2^{-16} \times 2^{-10}$	$1,0000; 1.11111111$
最小负数	$-2^{15} \times (1-2^{-10})$	$0,1111; 1.0000000001$

$-(1-2^{-10})$
 $= 2^{-10} - 1 < 0$
 1.11111111 要变 1.0000000001

机器零

- 当浮点数尾数为0时,不论其阶码为何值,都按机器零处理
- 当浮点数阶码等于或小于它所表示的最小数时,不论其尾数为何值,都按机器零处理

↓ 看作下溢,接近于0

如 $m=4, n=10$

当阶码和尾数都用补码表示时,机器零为

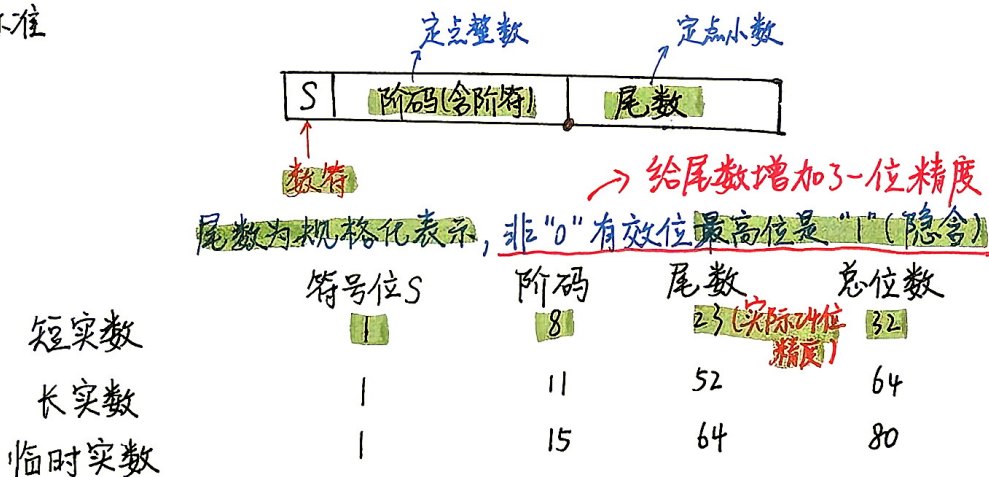
$x, xxx \times x ; 0.00 \dots 0 \quad 2^5 = 32 \text{ 种情况可表示 } 0$
 $\text{阶码} = -16 \rightarrow 1, 0000 ; x, xx \dots x \quad 2^{11} = 2048 \text{ 种情况可表示 } 0$

当阶码用移码,尾数用补码表示时,机器零为

$0, 0000 ; 0.00 \dots 0$

→ 有利于机器中“判0”电路的实现

IEEE 754 标准



与移码有区别,以32长度的IEEE 754标准的格式为例,

阶码长度为8位,那其移码偏移量应为 2^7

但在IEEE 754标准中,偏移量只有 $2^7 - 1$

原因: 其将阶码为全1的形式用来表示无穷大,与数符结合,可表示正/负无穷大.

阶码全0时表示0.

例. 若浮点数x的754标准存储格式为41360000H,求浮点数的十进制数值

解: 将16进制转为2进制: $0100\ 0001\ 0011\ 0110\ 0000\ 0000\ 0000\ 0000$
 ↑
 数符 (S) 阶码 (8位) 尾数 (23位)

∵ 阶码偏移量为 $2^7 - 1 = 127$ ∴ 指数 e (阶) = 阶码 - 127 = $1000010 - 1111111 = 11 = (3)_{10}$

∵ 有效位最高位“1”隐含 ∴ 尾数 $1.M = 1.0110110000000000000$

= 1.011011

$$\begin{aligned} \therefore x &= (-1)^S \times 1.M \times 2^e \\ &= + (1.011011) \times 2^3 \\ &= +1011.011 = (11.375)_{10} \end{aligned}$$

解: 首先将十进制转换为二进制

然后按 154 标准进行规格化

c. $10100.10011 = 1.010010011 \times 2^4$, 阶码 = 4

于是得到：

由于该数是正数 $\therefore S = 0$

$$\bar{E} = e + |2| = 131 = (10000011)_2$$

$$M = 0100|0011$$

最后得到的32位浮点数二进制存储格式为：

0 | 000001 | 0100 | 001 | 00 0000 0000 0000
 ↑
 S 8位 23位

$$= (41A4C000)_{16}$$

阶码: $1 \sim 254$

→ 真实阶码: $-126 \sim 127$

浮点数的解释(32位)

还原过后,
尾数部分没有高位隐含1

问：754 规格化表示范围？最大正数？最小负数？共能表示多少个数？

十进制数码的编码

用四位二进制代码的不同组合来表示一个十进制数码的编码方法，称为二-十进制编码。(BCD码)

依据每个二进制位是否有确定的位权，可将二-十进制编码分为有权码和无权码。

无权码中，用的较多的是余3码和格雷码(循环码)

在8421码的基础上，将每个代码加上“0011”形成，普通8421码的加法器仍能为余3码的加法器直接利用

相邻的两个代码只有一个二进制位的状态不同，有利于保证代码变换的连续性

主要有两种形式：

① 字符形式：一个字节存放一个十进制数位或符号位

0~9 + 个数字 ↓ 正、负号的ASCII编码值

例如，+123的编码是 2B 31 32 33 ， 占用4个连续的字节

-123的编码是 2D 31 32 33

由于其高4位不具备数值意义，主要用于非数值计算

② 压缩的十进制数形式：用一个字节存放两个十进制数位，既节省了存储空间，又便于完成十进制数的算术运算，其值用BCD码或ASCII码的低4位表示，

符号位也占半个字节并放在最低数字位之后

例如，用C表示正号，用D表示负号，并规定数字和符号位个数之和必须是偶数，否则在最高数字之前补一个0。

123 可表示为 12 3C (2个字节)

-12 可表示为 01 2D (2个字节)

非数值数据

目前国际上普遍采用的字符系统是7位的ASCII码

- 包括128个元素
- 二进制编码需要7位, 加1位偶校验位, 共8位1个字节

字符编码:

ASCII码、EBCDIC码

汉字的表示方法

机外码 机内码 点阵码

- ① 汉字输入码: ^{输入}将汉字转换成计算机能接收的编码
- ② 汉字内码: ^{内部处理}输入码进入计算机后被转换成这种形式进行处理
- ③ 汉字字形码: ^{输出}为显示/打印输出汉字, 需对汉字内码变换成这种形式

汉字的编码与输入

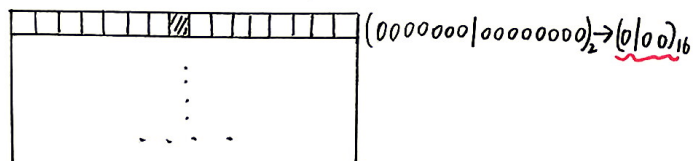
- ① 数字编码, 例如 国 - 2590 Σ - 0138 常用: 区位码
- ② 拼音码, 例如 GUO - 国
- ③ 字型编码 (如五笔字型)

汉字的存储

点阵码.

- ① 字型码 —— 用点阵组成的汉字的代码 —— 构成字库
- ② 机内码 —— 用于汉字的存储、交换、查询等

假设1个16×16的点阵



英文内码

0 | x x x x x x x x

汉字内码

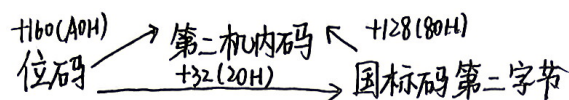
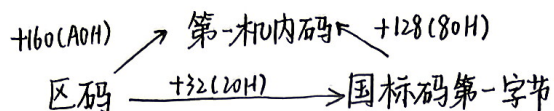
{ 1 | x x x x x x x x

1 | x x x x x x x x

↓
与国标码、区位码都是2个字节

汉字的存储

区位码、国标码与机内码的关系



例如：“中”字区位码为“5448”，其国标码为“5650(H)”

36 30

“国”字区位码为“2590”，其国标码为“397A(H)”

19 5A

汉字	国标码		汉字内码
中	5650(H) (<u>01010110</u> <u>01010000</u> B)	$\xrightarrow{+80H}$ 即“0”变“1”	11010110 11010000 B
国	397A(H) (<u>00111001</u> <u>01110101</u> B)		10111001 11110101 B
	computer 中 形式		